

The results of the static analysis mainly include:

- Function Definitions (Page 2)

- Available Actions (Page 2)

- Data/Control Dependencies (Page 2)

- Call Graphs (Page 3)

- Error-Propagation Path (Page 4).

Key prompts mainly include:

- EH-Database: Learning Error-Handling Strategies. (Page 5)

- Repair Agent: Fixing the Error-Handling Bug, Revising the Patch (Page 6)

- Validate Agent: Validating the Generated Patch (Page 7)

The static analysis results are used in the EH-Database, Repair Agent, and Validate Agent. We have underlined the results used in the prompt on pages 5-7.

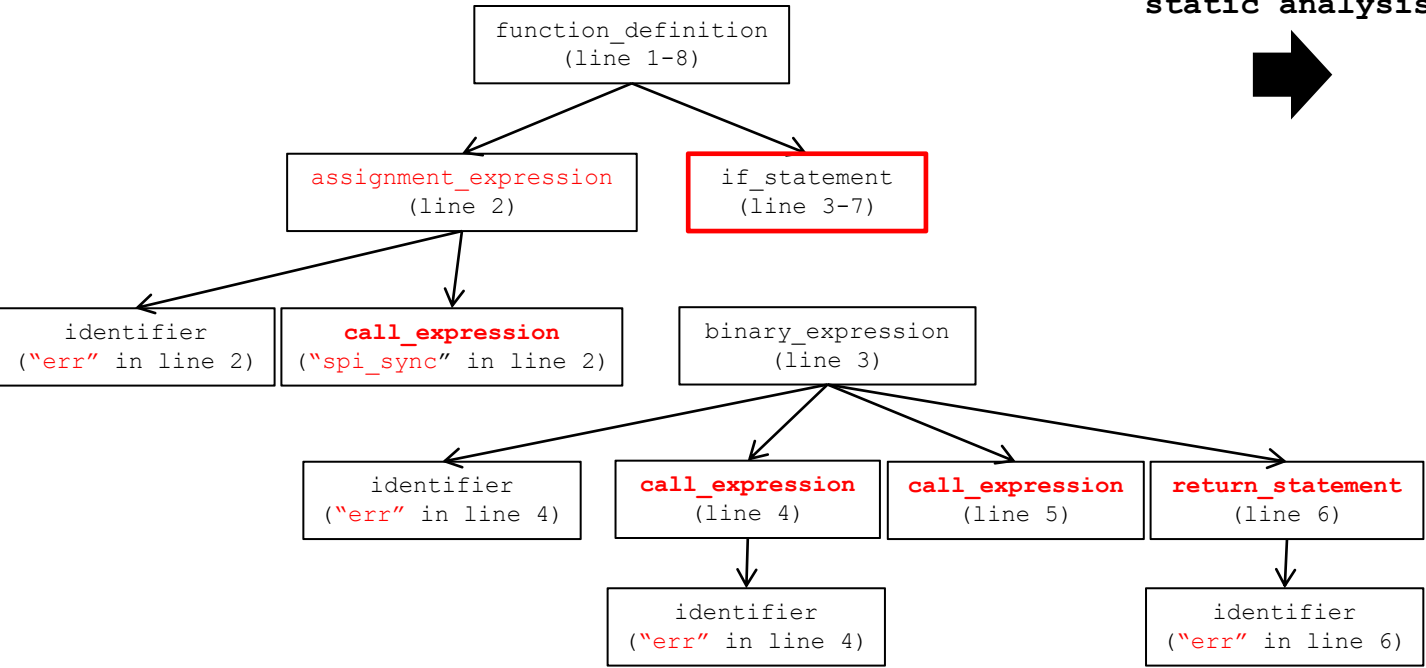
Source Code

```
1 static int max1111_read(struct device *dev, int channel){
  ...
2   err = spi_sync(data->spi, &data->msg);
3   if (err < 0) {
4       dev_err(dev, "spi_sync failed with %d\n", err);
5       mutex_unlock(&data->drvdata_lock);
6       return err;
7   }
  ...
8 }
```



tree_sitter

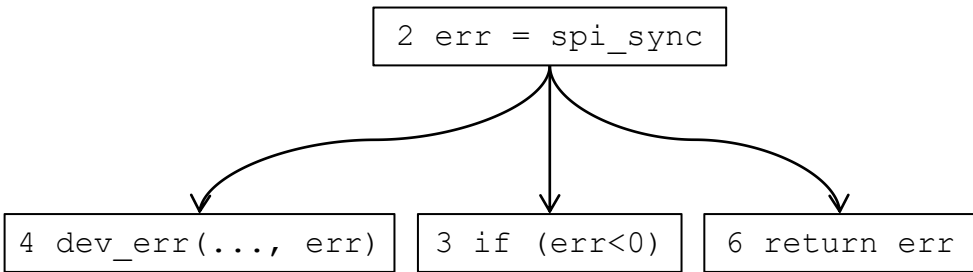
static analysis



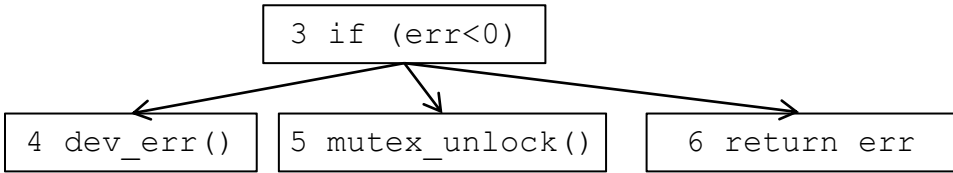
AST

"max1111_read" called "spi_sync", "dev_err", and "mutex_unlock"

Function Definition and Available Actions (Function Calls/Variables) can be obtained by analyzing nodes such as function_definition, call_expression, and identifier.



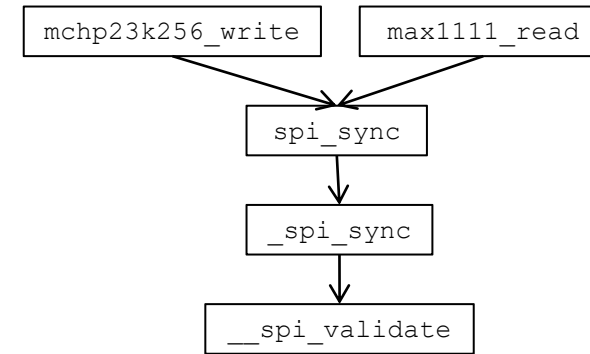
Data Dependencies can be constructed by analyzing the assign_statement ("err" and "spi_sync" in line 2), as well as the variable names ("err" in lines 2, 3, 4, 6).



Control Dependencies in a single function can be constructed by analyzing branching nodes (e.g., if_statement).

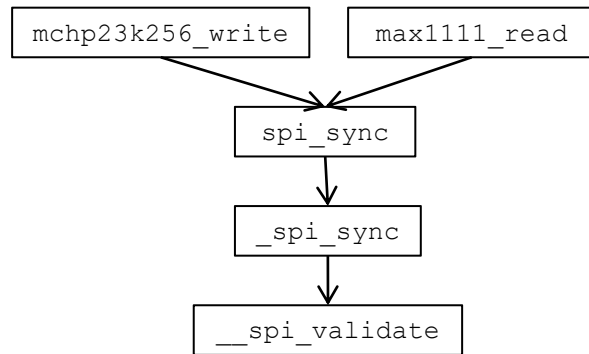
```
_spi_sync called __spi_validate  
spi_sync called __spi_sync  
mchp23k256_write called spi_sync  
max1111_read called spi_sync
```

Function call in each function



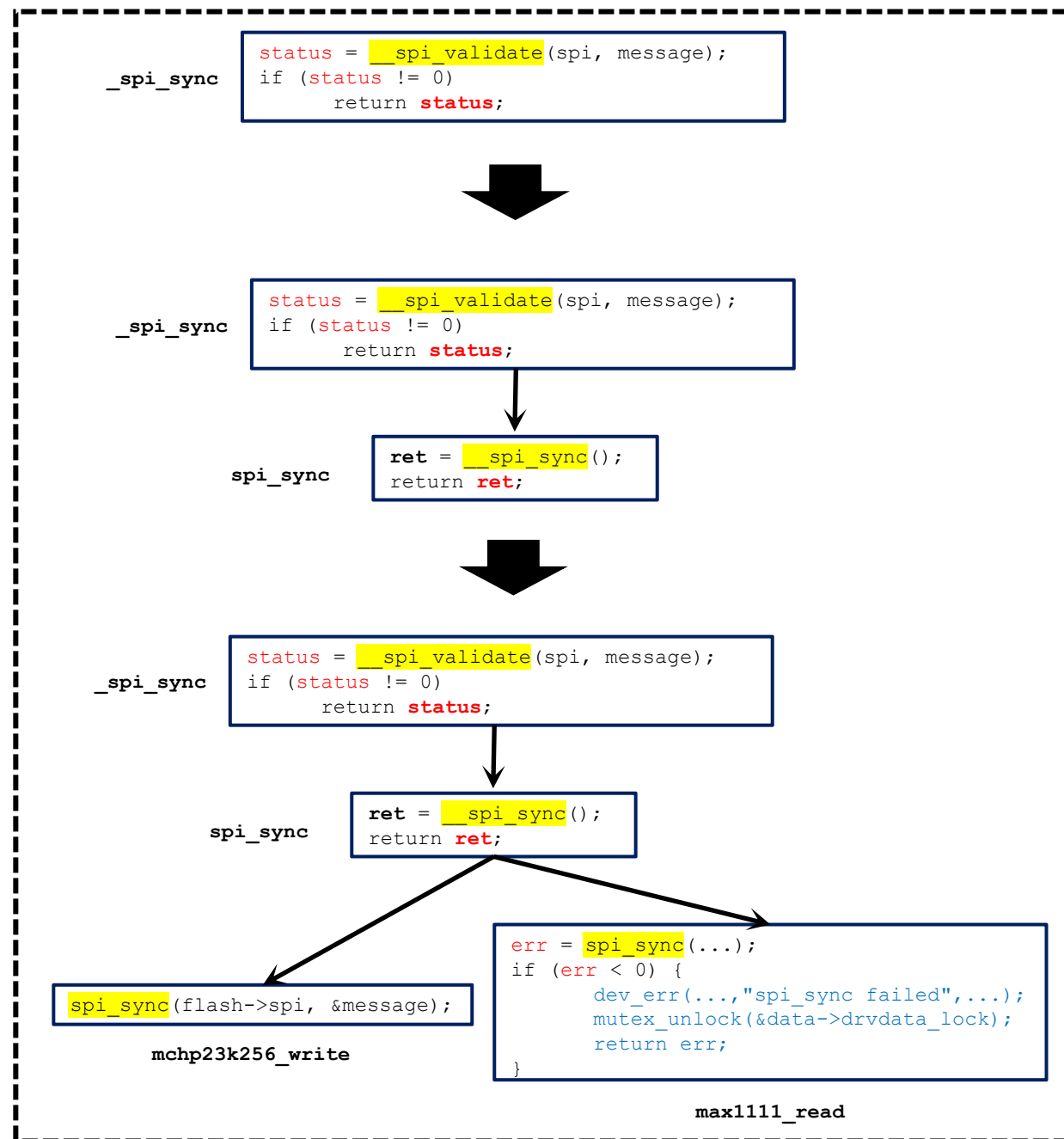
Call graph

Call Graph can be constructed by using the obtained function calls as edges.



Call graph

Inter-Procedural Static Analysis: Following the call graph, we splice the **error propagation path recursively** from leaf nodes ("spi_sync") to root nodes ("mchp23k256_write" and "max1111_read"). Each function will **only be analyzed once**, thus alleviating the search space explosion problem of inter-procedural static analysis.



Error Propagation Path Construction

Example

```
1.{buggy location}:
line 2 of function max1111_read

2.{error}:
spi_sync(data->spi, &data->msg);

3.{buggy function}:
max1111_read

4.Source code of buggy function:
Function definition of max1111_read

5.Source code of caller functions:
(Sliced code snippets according to
data/control flow dependencies)
"
static ssize_t show_adc(struct device
*dev,struct device_attribute *attr, char
*buf)
    int channel =
to_sensor_dev_attr(attr)->index;
    int ret;
    ret = max1111_read(dev, channel);
    if (ret < 0)
        return ret;
"
```

Learning Error-Handling Strategies

Instruction

Assume the role of an experienced software engineer, and **learn error-handling strategies** from the error handling for {bug location}. You will be provided with the source code of {buggy function} and its callers as contextual information. You should proceed according to the following steps:

1. **Analysis error Impact:**
How {error} is used by {buggy function}, and how it may affect the functionality of {buggy function} in case of an error.
How {buggy function} is used by its callers, and how {buggy function} might affect callers when it goes wrong.
2. **Analysis Handling Actions:**
List how {error} in the {buggy function} is handled: whether {buggy function} is stopped; what information is logged; which resources are cleaned up; which error code is returned.
3. **Analysis Relation Pairs:**
Correlate these impacts with handling actions, forming concise relationship pairs.

Example

```
Output: {
  "Impact": ["SPI transmission failure", "Cannot pass input data for caller functions"],
  "Handling Actions": ["Logging", "Resource cleanup", "Earllystop", "Propagate error"],
  "Relationship Pairs": [
    <"SPI transmission failure", ["Logging", "Resource cleanup", "Early stop"]>,
    <"Cannot pass input data for caller functions", ["Propagate error"]>
  ]
}
```

Constraints

1. If there are functions whose source code also needs to be analyzed but are not in the Contextual Information, output them in JSON format {"Function Retrieval": [function names]}
2. Focus only on the error produced by {error}.
3. Output in JSON format as shown in the Example.
4. If there is no corresponding action, leave the list empty.
5. "return;" should be outputted as "NULL"

Contextual Information

Source Code: [buggy function and caller functions]

Additional Information: [Information retrieved based on "Function Retrieval"]

Fixing the Error-Handling Bug

Instruction

Assume the role of an experienced software engineer. [{bug location} may lack error handling.]+ You will be provided with: [the source code of {buggy function} and its callers;]+ <impact, action> pairs demonstrate how other errors are handled; available actions that can be used for patch generation as contextual information. For each error, you should proceed according to the following steps:

- 1. **Analysis error Impact:**
How {error} is used by {buggy function}, and how it may affect the functionality of {buggy function} in case of an error.
How {buggy function} is used by its callers, and how {buggy function} might affect callers when it goes wrong.
- 2. **Predict Handling Actions:**
Refer to the <impact, action> of other errors, and whether {error} needs error handling based on its impact, and if so, what handling actions are needed.
- 3. **Generate Patch:**
If error handling is required, generate a patch for {error} based on the predicted handling actions and the available actions.

Example

```
{bug location}:{
  "Impact": ["SPI transmission failure", "Cannot pass input data for caller functions"],
  "Need Handling": True,
  "Relationship Pairs": [
    <"SPI transmission failure", ["Logging", "Resource cleanup", "Early stop"]>,
    <"Cannot pass input data for caller functions", ["Propagate error"]>
  ],
  "Handling Actions": ["Logging", "Resource cleanup", "Earllystop", "Propagate error"],
  "Patch": "+if (err < 0){
+dev_err(dev, "spi_sync failed with %d\n", err);
+mutex_unlock(&data->drvdata_lock);
+return err;}"
}
```

Constraints

- 1. Focus only on the error produced by {error}, and only use functions/return values from the Contextual Information.
- 2. If there are functions whose source code also needs to be analyzed but are not in the Contextual Information, output them in JSON format {"Function Retrieval": [function names]}
- 3. If there are resources that need to be cleaned up, but their cleanup function is not in the Contextual Information, output them in JSON format {"Cleanup Retrieval": [function names]}
- 4. Output in JSON format as shown in the Example.

Contextual Information

Source Code: [buggy function and caller functions]
Relation Pairs: [retrieved from functions in the same file or propagation path]
Available Actions: [retrieved from functions in the same file or propagation path]
Additional Information: [Information retrieved based on "Function Retrieval" and "Cleanup Retrieval"]

Example

- 1. **[]+**
When multiple functions are fixed collectively, they will be listed in "[]+"
- 2. **Available Actions:**
(Logging, dev_err, "Sliced code snippets")
- 3. **Validation Result**
(Natural language description returned by the Validate Agent or static check)
"+if (dev) +return; is checking the parameters of max1111_read, not related."
"free_dev is not in the contextual information"

Revising the Patch

Instruction

Assume the role of an experienced software engineer. {bug location} is missing error handling. You will be provided with: the source code of {buggy function}; available actions that can be used for patch generation as contextual information. It requires the following handling actions: {Predicted Handling Actions}. The last patch you generated is: {Incorrect Patch}. It had the following problems: {Validation Result}. Please revise the patch.

Example

```
Output: {
  "Patch": "+if (err < 0){
+dev_err(dev, "spi_sync failed with %d\n", err);
+mutex_unlock(&data->drvdata_lock);
+return err;}"
}
```

Constraints

Same

Contextual Information

Source Code: [buggy function]
Available Actions: [retrieved from functions in the same file or propagation path]
Additional Information: [Information retrieved based on "Function Retrieval" and "Cleanup Retrieval"]

Validating the Generated Patch

Instruction

Assume the role of an experienced software engineer. {bug location} is missing error handling. You will be provided with: the source code of {buggy function} as contextual information. It requires the following handling actions: {Predicted Handling Actions}. The last patch you generated is: {Incorrect Patch}. Please check whether all modifications are related to {error}.

Constraints

1. If a modification is not handling an error in {error}, it is considered irrelevant. Such as adding a check for the {buggy function} parameter.
2. Output in the following JSON format:

```
{  
  "Plausible": True/False,  
  "Problems": []  
}
```

Contextual Information

Source Code: [buggy function]